# 5.1 SYSTEM ARCHITECTURE — THE MENTAL MODEL

## Core Identity Shift

You are no longer writing a **scraper script**. You are building a **data acquisition system**.

A script answers:

> "Can I get this page?"

A system answers:

> "Can this run every day, survive failure, and be trusted without supervision?"

If the answer is not yes, the architecture is wrong.

---

## The Canonical Pipeline

```
Config → Fetch → Parse → Validate → Store → Resume
```

This is not a suggestion. This is a **contract**.

Each stage has **one responsibility** and **zero overlap**.

---

## Stage-by-Stage Mental Model

### 1. Config (Source of Truth)

**Purpose**

- Define *what* to scrape, never *how*.

**Must contain**

- URLs
- Headers
- Pagination rules
- Rate limits
- Output format
- Retry parameters (optional overrides)

**Rules**

- No scraping logic here
- No Python imports
- If config is invalid → system must **fail fast**

**Key Insight**

> If changing a client requires changing Python code, your architecture has failed.

---

## 2. Fetch (Untrusted I/O Boundary)

**Purpose**

- Retrieve raw data from the outside world.

**Input**

- Fully resolved request parameters from config

**Output**

- Raw HTML / raw JSON / raw response object

**Rules**

- No parsing
- No validation
- No business logic
- Network failures are expected, not exceptional

**Design Principle** Fetch is **retryable**, **isolated**, and **stateless**.

---

## 3. Parse (Extraction Only)

**Purpose**

- Convert raw responses into Python primitives.

**Input**

- Raw HTML / JSON

**Output**

- `dict` or `list[dict]` (still untrusted)

**Rules**

- No network calls
- No retries
- No schema enforcement
- Missing fields are allowed

**Key Insight** Parsing is *best-effort extraction*, not correctness enforcement.

---

## 4. Validate (Trust Boundary)

**Purpose**

- Decide what data is allowed to exist.

**Input**

- Raw parsed dicts

**Output**

- Schema-guaranteed objects

**Rules**

- This is where data can die
- This is where errors become visible
- Validation errors must be logged, not ignored

**Key Insight** Validation is the **firewall** between chaos and your system.

---

## 5. Store (Durability Layer)

**Purpose**

- Persist validated data safely.

**Input**

- Clean, validated records

**Output**

- CSV / JSON / Database rows

**Rules**

- Never store unvalidated data
- Writes must be atomic where possible
- Partial writes must be detectable

---

### 6. Resume (State & Recovery)

**Purpose**

- Make crashes irrelevant.

**Input**

- Persistent progress state

**Output**

- Correct continuation point

**Rules**

- Resume logic must not duplicate data
- Resume must survive power loss
- Resume must not require manual edits

---

## Non-Negotiable System Properties

### 1. Isolation

Each stage:

- Can be tested alone
- Can fail without corrupting others

### 2. Restartability

At any point:

- Kill process
- Restart
- System continues correctly

### 3. Observability

Every failure:

- Logged

- Timestamped
- Traceable to a stage

If you need to rerun with `print()` to debug, the system is broken.

---

## What This Architecture Prevents

- Silent data corruption
- "It worked yesterday" failures
- Client-specific spaghetti code
- One-off fixes that rot the system

This is the minimum architecture that **earns money reliably**.

---

# 5.2 FOLDER & PROJECT STRUCTURE — ENFORCING DISCIPLINE

## Why Structure Is Mandatory

Folder structure is not cosmetic. It is **behavioral enforcement**.

A bad structure allows bad decisions. A strict structure prevents them.

---

## Canonical Project Layout (Revisited)

```
scraper/
│
├── config/
│   ├── settings.yaml
│   └── sources.yaml
│
├── models/
│   └── schemas.py
│
├── core/
│   ├── fetch.py
│   ├── parse.py
│   ├── validate.py
│   └── retry.py
│
├── storage/
│   ├── save_csv.py
```

```
│   ├── save_json.py
│   └── save_db.py
│
├── state/
│   └── progress.json
│
├── logs/
│   └── scraper.log
│
├── main.py
└── requirements.txt
```

This structure **maps exactly** to the pipeline.

---

## Directory-Level Responsibilities

### `/config` — Behavior Without Code

**Purpose**

- Client-specific configuration
- Environment-specific overrides

**Rules**

- No Python logic
- Human-readable
- Version-controlled

**Anti-pattern** Hardcoded URLs inside `fetch.py`.

---

### `/models` — Data Contracts

**Purpose**

- Define what “valid data” means

**Contains**

- Pydantic schemas
- Shared enums
- Typed data structures

**Rules**

- No scraping logic
- No network logic

This folder defines **truth**, not behavior.

---

## `/core` — Pure System Logic

### `fetch.py`

- HTTP clients
- Browser automation
- Timeouts
- Headers

### `parse.py`

- HTML selectors
- JSON traversal
- Transformation to dicts

### `validate.py`

- Schema enforcement
- Error handling strategy
- Reject vs skip logic

### `retry.py`

- Retry policies
- Backoff strategies
- Status-code rules

**Rule** Each file must answer one question only.

---

## `/storage` — Output Adapters

**Purpose**

- Convert validated data into client-usable formats

**Rules**

- No scraping
- No parsing

- No validation

Storage is a **plug-in layer**. Same data, different outputs.

---

## `/state` — Memory for Crashes

**Purpose**

- Persist progress
- Enable resume

**Rules**

- Machine-written
- Never hand-edited
- Always flushed safely

If this folder is missing, your scraper is fragile.

---

## `/logs` — The Only Debugging Tool

**Purpose**

- Explain system behavior after the fact

**Rules**

- No `print()`
- Structured logs
- Append-only

If logs don’t explain failure, logs are useless.

---

## `main.py` — Orchestrator, Not a God File

**Purpose**

- Wire stages together
- Control execution flow

**Rules**

- No business logic
- No parsing
- No validation rules

`main.py` should read like a pipeline, not a novel.

---

## Structural Red Flags (Immediate Failure)

- Everything in `main.py`
- Parsing inside `fetch.py`
- Validation inside parsing
- Writing files directly from parsing
- Config values referenced without validation

Any of these means you are **not** Stage 5.

---

## Why Clients Pay for This

Clients do not pay for code. They pay for:

- Predictability
- Recoverability
- Trust

This structure is how you deliver those **by design**, not effort.

---

If you want notes for **5.3 (Config-Driven Scraping)** or **5.7 (Pydantic deep-dive)** at the same rigor, say the section number only.

# 5.3 CONFIG-DRIVEN SCRAPING (NO HARDCODING)

---

## Core Principle

**Code is permanent. Config is disposable.**

If a client change requires editing Python code, you have failed Stage 5.

---

## What “Config-Driven” Actually Means

Config is not just “URLs in YAML”. Config defines **scraping behavior** without touching code.

Your system must obey this rule:

> Same codebase + different config = different scraper.

---

## What Lives in Config (Non-Negotiable)

### 1. Source Definition

- Base URL
- Endpoint paths
- Query parameters
- Pagination strategy

### 2. Request Behavior

- Headers
- Cookies (if required)
- Auth tokens (via env vars, not plaintext)
- Timeout values

### 3. Pagination Rules

- Page numbers
- Offset/limit
- Cursor keys
- Stop conditions

### 4. Rate & Retry Hints

- Requests per second
- Retryable status codes
- Max retry attempts (optional override)

### 5. Output Rules

- Output formats (CSV / JSON / DB)
- File naming patterns
- Batch size for writes

---

## What Must NEVER Be Hardcoded

- URLs
- Page limits
- CSS selectors that change per client
- JSON keys that differ per source

- Rate limits
- Output paths

Hardcoding any of the above ties your system to **one client**.

---

## Example: Good vs Bad Thinking

### ❌ Script Thinking

```
url = "https://example.com/products?page=1"
```

### ☑ System Thinking

```
pagination:
  type: page
  param: page
  start: 1
  end: 1000
```

Code reads rules. Config decides behavior.

---

## Config Validation (Fail Fast or Die)

Config is **untrusted input**.

Before scraping starts:

- Validate schema
- Validate required fields
- Validate pagination strategy compatibility

If config is invalid:

- Abort immediately
- Log the reason
- Do not attempt scraping

Running with a bad config is worse than not running at all.

---

## Environment Separation (Dev vs Prod)

Same config **structure**, different values.

Typical split:

- `settings.dev.yaml`
- `settings.prod.yaml`

Differences:

- Rate limits
- Logging verbosity
- Output destinations
- Retry aggressiveness

Never branch logic in code like:

```
if env == "prod":
```

Environment affects **config only**, not logic.

---

## Mental Model to Lock In

Config answers:

- **What to scrape**
- **How far**
- **How often**
- **How strict**

Code answers:

- **How to fetch**
- **How to parse**
- **How to validate**
- **How to recover**

If those roles mix, complexity explodes.

---

# 5.4 LOGGING (PRODUCTION, NOT PRINT)

## Core Truth

If you cannot explain yesterday's failure **without rerunning**, your system is unprofessional.

Logging is not decoration. Logging is **post-mortem visibility**.

---

## Why `print()` Is Forbidden

`print()`:

- Has no severity
- Has no timestamp
- Has no persistence
- Cannot be filtered
- Cannot be correlated

A system that uses `print()` cannot be debugged after failure.

---

## What Logging Is Responsible For

Logs must answer these questions **after the fact**:

1. What was the system doing?
2. What request failed?
3. Why did it fail?
4. What did the system do next?
5. What data was affected?

If logs can't answer these, logging is insufficient.

---

## Log Levels (Use Them Correctly)

### INFO

- Request started
- Request completed
- Page successfully processed
- Resume point loaded

Used to reconstruct execution flow.

---

### WARNING

- Retry triggered
- Partial data missing

- Non-fatal validation failures
- Temporary rate limits

Warnings indicate **degradation**, not failure.

---

### ERROR

- Max retries exceeded
- Unrecoverable parsing failure
- Config validation failure
- Storage write failure

Errors indicate **loss of correctness**.

---

## Mandatory Log Contents

Every meaningful log entry must include:

- Timestamp
- Log level
- Source (module/function)
- Context identifier (page number, cursor, item id)
- Human-readable message

Optional but powerful:

- Request ID
- URL or endpoint name
- Retry attempt count

---

## Logging Scope by Pipeline Stage

### Fetch

- URL requested
- Status code
- Retry attempt
- Timeout / connection errors

### Parse

- Parsing start/end
- Selector failures (non-fatal)

- Empty responses

### Validate

- Validation success count
- Validation failure count
- Reason for rejection

### Store

- Rows written
- Output destination
- Write failures

### Resume

- Last checkpoint loaded
- New checkpoint saved

---

## Logging Strategy Rule

**Log events, not data.**

Bad:

- Logging full HTML
- Logging full JSON payloads

Good:

- Logging counts
- Logging identifiers
- Logging reasons

Logs must stay readable at scale.

---

## Silent Failure Is a System Bug

If any of these happen without a log:

- Retry occurs
- Data is skipped
- Validation fails
- Resume point changes

Then the system is broken.

Silence equals **untraceable corruption**.

---

## Mental Model to Lock In

Logs are not for you **now**. Logs are for you **later**, when:

- The client complains
- The system ran unattended
- The bug cannot be reproduced

If logs can reconstruct reality, you are operating at Stage 5.

---

If you want the same-level notes for **5.5 (Retry)** or **5.6 (Resume logic)**, say only the section number.

# 5.5 RETRY & FAULT TOLERANCE (TENACITY)

## Core Reality (Accept This First)

**Failure is normal.** Networks fail. Servers throttle. DNS breaks. Connections drop.

A scraper that crashes on failure is **amateur engineering**.

---

## Correct Mental Model

Retry logic is **not error handling**. Retry logic is **expected control flow**.

Your system must assume:

- Requests will fail
- Some pages will never succeed
- Completion ≠ 100% success

Goal:

> **Maximum correct data with minimum disruption**

---

## Where Retry Logic Belongs

Retry applies ONLY to:

- Network calls
- HTTP fetches
- Browser navigation

### Retry must NEVER apply to:

- Parsing
- Validation
- Storage

If parse fails repeatedly, retrying fetch is useless. If validation fails, retrying is dangerous.

---

## What You Retry On (Explicit Rules)

### Retry-worthy conditions

- Connection timeouts
- DNS failures
- HTTP 429 (rate limit)
- HTTP 5xx (server errors)

### Never retry

- HTTP 400
- HTTP 401 / 403 (auth / permission)
- HTML structure changes
- Validation errors

Retrying non-retryable failures **wastes time and hides bugs**.

---

## Exponential Backoff (Non-Optional)

Linear retry gets you blocked. Exponential backoff keeps you alive.

**Concept**

- Attempt 1 → immediate
- Attempt 2 → wait
- Attempt 3 → wait longer
- Stop at max attempts

This protects:

- Your IP

- The target server
- Your system's throughput

---

## Max Attempts (Hard Stop)

Retries must end.

Rules:

- Define a strict max attempt count
- Log final failure clearly
- Move on to next unit of work

A scraper that retries forever is **non-terminating software**.

---

## Partial Success Philosophy

This rule separates engineers from beginners:

> **80% correct data today is better than 0% tomorrow**

Your system must:

- Skip permanently failing pages
- Continue scraping remaining pages
- Record what failed and why

Stopping the entire run for one bad page is unacceptable.

---

## Retry + Logging Coupling

Every retry must be logged:

- Attempt number
- Reason
- Delay duration
- URL / page identifier

If retries happen silently, you will never know:

- How unstable the source is
- When throttling started
- Why runs are slow

---

## Failure Escalation Strategy

1. Attempt fetch
2. Retry on allowed failures
3. Exhaust retries
4. Log ERROR
5. Mark unit as failed
6. Continue system execution

Crashing the system is the **last resort**, not default behavior.

---

## Lock-In Mental Rule

Retry logic exists to **protect progress**, not guarantee success.

If retry logic hides errors → bad If retry logic enables completion → correct

---

# 5.6 RESUME LOGIC (CRITICAL FOR MONEY)

## The Non-Negotiable Truth

Any scraper that cannot resume is **unsellable**.

Clients do not care why it crashed. They care whether progress was lost.

---

## The Core Problem

Scraping jobs are long-running:

- Thousands of pages
- Hours of execution
- Unreliable networks

Without resume:

- One crash = full restart
- Time wasted
- Data duplicated
- Trust destroyed

---

## Resume Logic Definition

Resume logic means:

> The system can restart and **continue from the last known correct state** without manual intervention.

No flags. No hand-editing files. No custom restarts.

---

## What Must Be Stored as State

Only store **minimal, sufficient progress**.

Examples:

- Last successful page number
- Last processed cursor
- Last item ID
- Timestamp of last write

Never store:

- Raw HTML
- Entire datasets
- Temporary variables

State must be:

- Small
- Machine-written
- Durable

---

## Progress Checkpointing (How It Works)

Checkpoint after:

- Successful page fetch + parse + validate + store

Never checkpoint:

- Before validation
- Before storage completes

Checkpointing invalid or partial data corrupts resumes.

---

## Idempotency (Critical Concept)

Resume logic only works if re-runs are **safe**.

Idempotent run means:

- Reprocessing the same page does not duplicate data
- Rewriting the same output does not corrupt results

Techniques:

- Deterministic file names
- Deduplication keys
- Append-safe writes

If resume causes duplication, resume is broken.

---

## Resume Granularity

Too coarse:

- Resume every 1,000 pages → data loss on crash

Too fine:

- Resume every item → slow, noisy

Correct:

- Resume at **logical units** (page, batch, cursor)

Granularity must balance:

- Safety
- Performance
- Simplicity

---

## Resume + Logging Coupling

Every resume event must be logged:

- Loaded checkpoint
- Resume position
- Reason for resume (restart, crash, manual rerun)

If resume happens silently, debugging becomes impossible.

---

## Crash Scenarios You Must Survive

Your system must recover from:

- Power loss
- Process kill
- Network outage
- Rate-limit lockout
- Machine reboot

If any of these require restarting from zero, resume logic is insufficient.

---

## Resume Is Not Optional Logic

Resume is not a “nice feature”.

Resume is:

- Reliability
- Cost control
- Professional credibility

A scraper without resume is a **toy**, regardless of how fast it is.

---

## Final Lock-In Rule

Retry protects **requests**. Resume protects **time**.

If you lose time on failure, you are not operating at Stage 5.

Below are **Stage-5 closing notes** for **5.9 and 5.10**. This is where theory ends and **professional credibility begins**.

---

# 5.9 END-TO-END PRACTICE TASK (MANDATORY)

## Purpose of This Task (Do Not Misread It)

This task is **not practice**. This task is a **qualification test**.

If you cannot build this system **end-to-end without babysitting**, you are not freelance-ready.

---

## What You Are Actually Proving

This task proves five things simultaneously:

1. You can design a long-running system
2. You can survive failure without panic
3. You can guarantee data correctness
4. You can operate unattended
5. You can hand results to a client confidently

Anything less is partial competence.

---

## Non-Negotiable System Requirements (Reinterpreted)

### 1. Scrape 500+ Items

This is a **stress test**, not a number target.

Purpose:

- Long runtime
- High chance of network issues
- Memory pressure
- Pagination exposure

If your system only works for 50 items, it is untested.

---

### 2. Config-Driven

**Hard rule**:

- Changing target = editing config only

Test:

- Duplicate config
- Change URL + pagination
- Run same code

If code changes → system fails qualification.

---

### 3. Logs Every Request

Logs must allow you to answer:

- Which page was fetched
- When
- With what result
- What failed
- What retried
- What was skipped

If logs cannot reconstruct the run, the run never happened.

---

## 4. Retries on Failure

Required behavior:

- Temporary failures → retried
- Permanent failures → logged and skipped
- System continues

Disallowed behavior:

- Infinite retry
- Silent retry
- Retry parse or validation

Retry logic must be **predictable**, not hopeful.

---

## 5. Resume After Crash

You must simulate failure.

Kill the process:

- Mid-pagination
- Mid-batch
- During retries

Restart and verify:

- No duplicate data
- No skipped valid pages
- Resume point correct

If resume works only in theory, it does not work.

---

## 6. Validation via Pydantic

Validation must:

- Reject malformed records
- Log rejection reasons
- Preserve valid records

Manual cleaning after export is forbidden.

If you “fix rows by hand”, the system is incomplete.

---

### 7. Outputs CSV + JSON

Purpose:

- CSV → client consumption
- JSON → machine integration

Both must be:

- Generated from validated data
- Deterministic
- Reproducible

If rerunning produces different structure, trust is broken.

---

### 8. Zero Manual Fixes

This is the **hardest requirement**.

Zero manual fixes means:

- No editing output files
- No rerunning “just one page”
- No temporary hacks
- No patching data post-hoc

If a human must intervene, automation has failed.

---

## The Only Acceptable Failure Mode

The only acceptable failure is:

> “Some records were skipped and clearly logged, but the system completed correctly.”

Any other failure means Stage 5 is incomplete.

---

## Internal Audit Checklist (Use This)

Before declaring completion, verify:

- Can I explain every skipped record?
- Can I explain every retry?
- Can I explain total row count difference?
- Can I rerun safely without cleanup?
- Can I change source via config only?

If any answer is no, do not move on.

---

# 5.10 DONE CRITERIA (OBJECTIVE, NOT EMOTIONAL)

## Why Feelings Are Irrelevant Here

Confidence is not evidence. Completion is evidence.

Stage 5 is passed by **observable system behavior**, not belief.

---

## Criterion 1 — Safe Restart

**Test**

- Kill process randomly
- Restart
- Observe continuation

**Pass condition**

- No duplication
- No loss
- No manual flags

If restart requires thinking, it is unsafe.

---

## Criterion 2 — Automatic Invalid Data Detection

**Test**

- Break a selector
- Change site structure
- Introduce malformed data

**Pass condition**

- Invalid data rejected
- Reasons logged
- System continues

If corruption reaches output, validation failed.

---

## Criterion 3 — Logs Explain Failure Without Reruns

**Test**

- Do not rerun
- Read logs only

**Pass condition**

- You can explain:
    - What failed
    - Where
    - Why
    - Impact

If rerun is required to understand failure, observability failed.

---

## Criterion 4 — New Client = New Config

**Test**

- New target site
- Same code
- New config

**Pass condition**

- System runs
- Output valid
- Logs meaningful

If you hesitate to accept a new client, architecture is weak.

---

## Criterion 5 — Unattended Trust

This is the ultimate test.

Ask yourself:

> "Would I let this run overnight on a paid contract?"

If the answer is not an immediate yes, Stage 5 is not complete.

---

## What Changes After Stage 5

Before Stage 5:

- You debug scripts
- You fear crashes
- You babysit runs
- You hesitate to charge

After Stage 5:

- You operate systems
- You expect failure
- You rely on logs
- You sell reliability

This is the line between **coding** and **engineering**.

---

## Final Lock-In Statement

Stage 5 is complete when:

- Failure is boring
- Restart is routine
- Data is trusted
- Clients are confident
- You stop thinking like a scraper

At that point, scraping becomes **infrastructure**, not effort.